

### Clase Password

```
public class Password {  
    // CLAVE única  
    private final int CLAVE = 67294358;  
    // Contraseña encriptada  
    private int passwordEncriptada;  
    public Password(int password) {  
        this.passwordEncriptada =  
            encriptarDesencriptar(password);  
    }  
    protected int encriptarDesencriptar(int password) {  
        return password ^ CLAVE;  
    }  
    public void guardarContrasena() {  
        System.out.println("La contraseña se ha guardado como "  
            + passwordEncriptada);  
    }  
    public boolean iniciarSesion(int password) {  
        int passwordEncriptadaIntento =  
            encriptarDesencriptar(password);  
        if (passwordEncriptadaIntento  
            == passwordEncriptada) {  
            System.out.println("Bienvenido");  
            return true;  
        } else {  
            System.out.println(  
                "Error al iniciar sesión. Contraseña incorrecta.");  
            return false;  
        }  
    }  
}
```

```
}  
}
```

### *Clase PasswordAmpliada*

```
public class PasswordAmpliada extends Password {  
    private int passwordDesencriptada;  
    public PasswordAmpliada(int password) {  
        super(password);  
        this.passwordDesencriptada = password;  
    }  
    @Override  
    public void guardarContrasena() {  
        System.out.println(  
            "La contraseña se ha guardado como "  
            + passwordDesencriptada);  
    }  
}
```

### *Clase Main*

```
public class Main {  
    public static void main(String[] args) {  
        int password = 123456;  
        Password password1 =  
            new Password(password);  
        password1.guardarContrasena();  
        // Login correcto  
        password1.iniciarSesion(123456);  
        // Login incorrecto  
        password1.iniciarSesion(111111);  
        // Clase hija  
        PasswordAmpliada password2 =
```

```
        new PasswordAmpliada(987654);

        password2.guardarContrasena();

        password2.iniciarSesion(987654);

        password2.iniciarSesion(222222);

    }

}
```

## **RESPUESTAS TEÓRICAS**

### **1. ¿Has creado algún atributo como final? ¿Por qué?**

Sí, he creado el atributo CLAVE como final porque su valor nunca debe cambiar.

Representa una clave fija usada para encriptar y desencriptar las contraseñas.

```
private final int CLAVE = 67294358;
```

### **2. ¿Todos los métodos deben ser accesibles?**

No.

El método encriptarDesencriptar() no debería ser público porque es un método interno de la clase y no necesita ser utilizado desde fuera.

Por eso se ha puesto como protected, para que solo pueda usarse desde la clase o sus hijas.

### **3. ¿Sobreescribir guardarContraseña es correcto?**

Técnicamente sí, porque Java permite sobreescribir métodos.

Pero no sería recomendable en este caso porque en la clase hija se está mostrando la contraseña desencriptada, lo cual rompe la seguridad del programa.

Una posible solución sería declarar el método como final en la clase padre:

```
public final void guardarContrasena()
```

Así ninguna clase hija podría modificar su comportamiento.

## **BLOQUES ESTÁTICOS**

### **¿Qué imprimirá el código?**

1º bloque. Inicializa el nombre

2º bloque estático de inicialización

Entra en el constructor

Entra en el método

Nombre: Patricia

Valor de mystr: Block2